Studio Nx

JS 287 — 서버 클립 노트 불러오기

2026-10-19

JavaScript 287 — 서버 클립 노트 불러오기 (`sequencer-roll.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-roll.js`
작업	S1-10a 클립·연결 저장: 서버 클립 노트 불러오기, 패턴 길이 반영, 롤→시퀀서 노트 저장 경로 연결 (FR-2196 클립·패턴)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 286 패턴 만들기와 롤 노트 저장, 다음 288 (연결)

1. 하는 일

피아노롤을 MIDI 클립에서 열 때, 서버가 준 `clip.__notes` 와 `clip.__patternBars` 를 읽어 롤에 채웁니다. `openRoll` 은 서버 클립이면 패턴 길이를 `setPatternBars` 로 반영하고, 노트를 `loadClipNotes` 로 tick → 마디 단위로 변환해 배치합니다. `loadClipNotes` 는 기존 노트를 지우고, `placeNote`/`renderVelocity`/`updateSelectedInfo` 로 화면을 갱신합니다. 또한 롤 노트가 바뀌면 `emitRollNotes` 가 `sq:roll-notes` 이벤트를 발행해 `sequencer.js` 의 `initClipPreviews` 가 `setClipNotes`, `clip.__notes` 갱신, `saveClipNotes(clip, notes)` 를 통해 `PUT /patterns/{id}/notes` 로 저장하게 이어집니다.

```
MIDI 클립 더블클릭
        ↓
sequencer.js initTracks / 편곡 → openRoll(clip)
        ↓
openRoll
  ├ clip.__patternBars → setPatternBars (격자·끝 표시에 패턴 길이 반영)
  └ clip.__notes → loadClipNotes
        ↓
tick → bars 변환 (start_tick / TICKS_PER_BAR, length_tick / TICKS_PER_BAR)
        ↓
노트 요소 생성 · placeNote · renderVelocity · updateSelectedInfo
        ↓
롤에서 편집 (추가/이동/리사이즈/삭제)
        ↓
initRoll.onChange → emitRollNotes → CustomEvent 'sq:roll-notes' { clip, notes }
        ↓
sequencer.js initClipPreviews → saveClipNotes → PUT /patterns/{id}/notes (400ms debounce)
```

2. 언제 쓰나

쓰는 곳	
편곡 타임라인	MIDI 클립을 더블 클릭하면 `openRoll(clip)` 이 불립니다.
`sequencer-devices.js` (clipElement)	서버 클립을 그릴 때 `el.__patternBars` 와 `el.__notes` 를 채웁니다.
`sequencer.js`	`sq:roll-notes` 이벤트를 듣고 편곡 미리보기를 갱신하며 노트를 저장합니다.
피아노롤 화면	`loadClipNotes` 가 롤 노트 목록을 다시 채웁니다.

3. 코드

3.1 openRoll: 서버 클립 값 불러오기

```
/** 피아노롤 열기 (아래 칸) — clip 이 있으면 제목을 그 클립으로 */
export function openRoll(clip) {
  const bottom = document.getElementById('bottom');
  bottom.hidden = false;
  bottom.querySelectorAll('.sq-panel').forEach((p) => { p.hidden = p.id !== 'panel-roll'; });
  R.clip = clip || null;
  if (clip) {
    document.getElementById('roll-title').textContent = `🎹 Piano roll — ${clip.querySelector('.sq-clip_
_name').textContent.replace(/^目\s*/, '')}`;
    const track = clip.closest('.sq-track');
    const name = track?.querySelector('.sq-track__name b')?.textContent.trim();
    const inst = track?.querySelector('.sq-track__name small')?.textContent.replace(/^.*?🎹\s*/, '').tri
m();
    if (name) document.getElementById('roll-track').textContent = `${name} · ${inst ?? ''}`;
    // 서버에서 온 MIDI 클립이면 그 패턴의 노트를 롤에 채웁니다 (S1-10a FR-2196)
    if (clip.__patternBars) setPatternBars(clip.__patternBars);
    if (clip.__notes) loadClipNotes(clip.__notes);
  }
  requestAnimationFrame(() => {
    applyBarWidth();
    if (!R.opened) { R.opened = true; scrollToNotes(); }
    syncLane();
    requestDraw();
  });
}
```

- `clip.__patternBars` 가 있으면 `setPatternBars` 로 롤의 패턴 길이를 지정합니다.
- `clip.__notes` 가 있으면 `loadClipNotes` 로 서버 노트를 롤에 채웁니다.
- 제목/트랙명은 기존 클립 이름/트랙 정보를 읽어 표시합니다.

3.2 setPatternBars

```
/** 패턴 길이(마디)를 반영 — 격자·끝 표시가 패턴 길이에 맞춥니다 (S1-10a) */
function setPatternBars(bars) {
  if (!R) return;
  const n = Math.max(1, Math.round(bars));
  R.patternBars = n;
  R.roll.style.setProperty('--roll-pattern-bars', n);
  requestDraw();
}
```

- 패턴 길이를 롤 상태에 저장하고 CSS 변수 `--roll-pattern-bars` 에 반영합니다.
- 격자/끝 표시가 해당 길이에 맞춰 보이도록 `requestDraw` 를 호출합니다.

3.3 loadClipNotes

```
/**
 * 서버 패턴의 노트를 롤에 채우기 (S1-10a FR-2196) — 지금 있는 노트(반투명 안내 노트는 남김)를 지우고,
```

```
 * 노트를 tick → 마디로 바꿔 놓습니다. 여기서 고치면 'sq:roll-notes' 로 알려 sequencer.js 가 저장합니다.
 * @param {object[]} notes  {pitch, start_tick, length_tick, velocity}
 */
function loadClipNotes(notes) {
  if (!R) return;
  R.notes.querySelectorAll('.sq-note:not(.is-ghost)').forEach((n) => n.remove());
  const frag = document.createDocumentFragment();
  (notes || []).forEach((nt) => {
    const el = document.createElement('span');
    el.className = 'sq-note';
    el.dataset.pitch = String(nt.pitch);
    el.dataset.start = String(round((nt.start_tick || 0) / TICKS_PER_BAR));
    const len = round((nt.length_tick || TICKS_PER_BAR / 4) / TICKS_PER_BAR);
    el.dataset.length = String(len > 0 ? len : 0.25);
    el.dataset.vel = String(nt.velocity || 100);
    placeNote(el);
    frag.append(el);
  });
  R.notes.append(frag);
  renderVelocity();
  updateSelectedInfo();
}
```

- 서버의 `start_tick`, `length_tick` 을 `TICKS_PER_BAR` 로 나눠 마디 단위(`dataset.start`, `dataset.length`)로 변환합니다.
- 안내용 반투명 노트(`.is-ghost`)는 남겨둡니다.
- `placeNote` 로 위치/길이/음높이 정보를 CSS 변수에 반영하고, `renderVelocity`/`updateSelectedInfo` 로 보조 영역을 갱신합니다.

3.4 롤 노트 발행 (emitRollNotes)

```
/** 지금 피아노롤의 노트 전부를 저장용 모양(tick 단위)으로 모은다 (요구사항 1, 2026-10-17) */
function rollNotes() {
  return [...R.notes.querySelectorAll('.sq-note:not(.is-ghost)')].map((n) => ({
    pitch: Number(n.dataset.pitch),
    start_tick: Math.round(Number(n.dataset.start) * TICKS_PER_BAR),
    length_tick: Math.round(Number(n.dataset.length) * TICKS_PER_BAR),
    velocity: Number(n.dataset.vel) || 100,
  }));
}

/** 노트가 바뀔 때마다 알린다 — sequencer.js 가 듣고 그 MIDI 클립의 미리보기를 다시 그린다 */
function emitRollNotes() {
  if (!R || !R.clip) return;
  document.dispatchEvent(new CustomEvent('sq:roll-notes', {
    detail: { clip: R.clip, notes: rollNotes(), bars: R.patternBars },
  }));
}
```

- 롤에 있는 일반 노트를 tick 단위로 다시 변환해 서버 형식으로 모읍니다.
- `sq:roll-notes` 커스텀 이벤트로 발행해 `sequencer.js` 가 저장과 미리보기 갱신을 이어 받습니다.
- 이벤트 디테일에 `clip` 을 포함해 어떤 클립의 패턴인지 정확히 식별합니다.

3.5 onChange 에 이벤트 발행 연결

```
// 변화 알림 — 사용자 onChange 뒤에 편곡의 클립 미리보기가 다시 그려지도록 알린다 (요구사항, 2026-10-17)
const userChange = onChange;
R.onChange = (change) => {
  if (typeof userChange === 'function') userChange(change);
  emitRollNotes();
};
```

- 피아노롤에서 노트가 추가/이동/리사이즈/삭제될 때마다 `onChange` 가 호출되고, 그 직후 `emitRollNotes` 로 현재 노트 목록을 발행합니다.
- 이렇게 해서 롤 편집 → 클립 미리보기 → 서버 저장까지 한 줄의 경로로 이어집니다.

4. 왜 이렇게

선택	이유
`clip.__notes` 와 `clip.__patternBars` 를 DOM 요소에 보관	서버 클립 정보를 편집창(롤)을 열 때 즉시 불러와 사용할 수 있게 합니다. `sequencer-devices.js` 의 `clipElement` 이 서버 데이터에서 이 값을 채우고, 롤은 그것만 읽습니다.
ghost 노트는 로드 시 지우지 않음	안내용 노트(`.is-ghost`)는 편집 보조용이므로 서버 노트와 겹치지 않게 일반 노트만 제거합니다.
tick ↔ bars 변환을 명시적으로 함	서버는 tick 단위, 롤은 마디 단위로 작업합니다. 로드 시에는 tick → bars, 저장 시에는 `rollNotes()` 가 bars → tick 으로 되돌립니다. 이 변환은 `TICKS_PER_BAR` (960)에 의존합니다.
`emitRollNotes` 가 변경 직후 호출	롤 노트 편집은 드래그 중에도 여러 번 일어나지만, `initRoll` 내부의 편집 경로에서 노트가 화면에 배치된 뒤 `onChange` → `emitRollNotes` 순으로 호출되도록 연결해, 미리보기와 저장 타이밍을 일관되게 만듭니다.
`patternBars` 를 CSS 변수로 노출	롤의 격자와 끝 마커가 패턴 길이에 맞게 그려지도록 CSS에서 참조할 수 있게 합니다.
클립 제목/트랙명을 롤 헤더에 표시	어떤 클립의 패턴을 편집하는지 사용자가 바로 알 수 있게 합니다.

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 항목을 확인합니다.

- 서버에서 불러온 MIDI 클립을 더블 클릭하면 피아노롤이 열리고, 그 클립의 노트가 롤에 정확히 표시됩니다.
- 패턴 길이가 긴 클립은 `setPatternBars` 로 롤 길이가 맞춰집니다.
- 롤에서 노트를 추가하면 `sq:roll-notes` 이벤트가 발행되고, 편곡 화면의 해당 MIDI 클립 미리보기에 막대가 즉시 그려집니다.
- 롤에서 노트를 이동/리사이즈/삭제해도 미리보기와 저장(`PUT /patterns/{id}/notes`)이 이어집니다.
- 롤을 닫았다가 같은 클립을 다시 열면, 마지막으로 저장된 노트 상태가 그대로 불러와집니다.
- 테스트 14/14 통과. 스크린샷: `docs/screenshots/2026-10-19_s1-10a_clips_connections/`.